

IILM UNIVERSITY, GREATER NOIDA

Department of Computer Science and Engineering

A PROJECT REPORT ON

NetSage AI

An AI-Assisted Network Troubleshooting System with Human Review



Submitted in partial fulfilment of the requirements for the
Cisco Networking Academy — Introduction to Modern AI / Applied AI Internship Programme

Submitted by

Yash Yadav

Roll No.: 2410030788

Section 3CSE11

IILM University, Greater Noida

2025 – 2026

Certificate / Approval Page

It is to certify that the report on the project titled "NetSage AI: An AI Assisted Network Troubleshooting System with Human Verification" is an authentic document which describes the work done by Purnima Giri (Roll No.: 2410030768), Section 3CSE11, IILM University, Greater Noida, as part of Cisco Networking Academy / Introduction to Modern AI course.

The above stated work has been done under my/our supervision and, to the best of my/our knowledge, the above said work is the independent work of the student.

Faculty Guide / Supervisor Signature

Name:

Designation:

Date:

Head of Department Signature

Name:

Date:

Student Declaration

I, Purnima Giri, Roll No. 2410030768, Section 3CSE11, IILM University, Greater Noida, solemnly declare that the project report titled “NetSage AI: An AI-Assisted Network Troubleshooting System with Human Review” presented by me is an original and genuine report regarding the work done by me on the Cisco Networking Academy / Introduction to Modern AI course.

Further, I solemnly declare that this report has not been submitted in whole or in part to any other institution/university for the award of any degree/diploma/certificate. The sources of information, data and reference material used for the preparation of this report have been acknowledged accordingly.

Further, I do hereby acknowledge that, as disclosed clearly in the respective chapters, the live Large Language Model (LLM) API integration which was supposed to be implemented in the diagnosis engine could not be accomplished due to API quota restriction, and the submission hereof employs the diagnosis engine using mock/rule-assisted mechanism as an alternate honest approach to the same.

Name: Purnima Giri

Roll No.: 2410030768

Date:

Place: Greater Noida

Acknowledgement

It is my pleasure to convey my sincere thanks to my guide faculty and the Department of Computer Science & Engineering, IILM University, Greater Noida for their consistent support and motivation during the entire process of this project.

I am also thankful to the Cisco Networking Academy for providing the Introduction to Modern AI curriculum and the applied AI and networking problem statement that formed the foundation of this project. The structured combination of networking fundamentals and applied artificial intelligence concepts significantly shaped the design of NetSage AI.

I extend my appreciation to my peers and mentors who provided valuable feedback during the design, implementation, and testing phases of this project, and to my family for their patience and encouragement during the course of this work.

Finally, I acknowledge that this project represents a learning exercise, and any limitations discussed transparently in this report — including the mock diagnosis engine used in place of a live AI API — are presented as opportunities for future improvement rather than shortcomings to be concealed.

Purnima Giri

Abstract

Junior network engineers frequently understand individual Cisco commands and configuration constructs in isolation but struggle to connect an observed network symptom with its underlying root cause. A single symptom, such as a PC failing to communicate with a server despite receiving a valid IP address, may originate from any of several fault domains — VLAN misconfiguration, incorrect default gateway assignment, DHCP failure, DNS failure, routing misconfiguration, access control list (ACL) restrictions, network address translation (NAT) errors, or wireless configuration issues. This project, NetSage AI, addresses this diagnostic gap by proposing and implementing an AI-assisted network troubleshooting system built around a mandatory human-in-the-loop review process.

Input to the system can be in one of the following three types for each trouble-shooting case: description of the network symptom in text form, topology notes describing how the network should be designed, and show-command output in Cisco format. Then the input goes to a deterministic Python rule checker which identifies any anti-pattern of configuration that is well-defined (such as interface being administratively down, VLAN not being in trunk, or implicit deny condition in ACL), followed by a diagnosis engine that gives a probable root cause, associated OSI layer, a confidence score, evidence, suggested next troubleshooting command, and instructions on how to fix the issue. It is very important to note that any diagnosis made by the system cannot be considered final since it has to be approved/rejected/edited by a person and the decision made has to be documented.

The technology stack includes Python (main logic), Streamlit (web interface), Pandas (data manipulation). At the moment, we use CSV files for persistent storage of our data during the current phase of development. Our dataset contains 36 trouble-shooting cases related to 8 networking fault categories: VLAN, Default Gateway, DHCP, DNS, Routing, ACL, NAT, and Wireless Networking. They correspond to OSI Layers 1 to 7. The original architecture was supposed to call an API from an OpenAI-compatible LLM, however, during development and testing, the API call generated an HTTP 429 rate-limit error because of lack of sufficient account quota and thus could not be shown in action. As a result, the diagnosis engine was refactored into a mock/rule-assisted engine with the same intended workflow, input/output

contract, and JSON schema so that in the future it can be easily replaced with an actual call to an LLM.

The methodology followed an iterative, prototype-based development approach across dataset design, rule-checker implementation, diagnosis-engine scaffolding, human-review-workflow implementation, and Streamlit application assembly. The resulting application exposes four modules — Home, Diagnose Network, Human Review, and Dashboard — allowing a user to select a case, run rule-based and mock-AI diagnosis, submit a human review decision, and view aggregate statistics of the dataset. Testing was carried out at the level of the dataset, rule checker, diagnosis engine, human-review logging, and Streamlit interface, using representative demonstration scenarios; results are reported honestly as either observed outcomes of the implemented rule-based logic or as proposed/demonstration test scenarios where a live AI evaluation was not possible.

This report presents the complete design, implementation, and evaluation of NetSage AI, together with an honest discussion of its current limitations — including its reliance on a simulated rather than live AI backend, its CSV-based storage, and its conceptual (rather than automated) integration with Cisco Packet Tracer. The report further outlines a future scope encompassing real LLM integration, retrieval-augmented generation (RAG) over Cisco documentation, database migration, multi-reviewer support, and AI-human agreement analytics. The main value of this research is that instead of trying to claim an autonomous AI troubleshooter, it offers a responsible approach to using AI technologies in network troubleshooting by humans.

Keywords: Artificial Intelligence, Network Troubleshooting, Cisco, OSI Model, Human-in-the-Loop, Responsible AI, Rule-Based Systems, Streamlit, Python, VLAN, Packet Tracer.

Keywords

Artificial Intelligence (AI); AI Application; Troubleshooting Networks; Cisco Networking Academy; Packet Tracer; OSI Model; VLAN; DHCP; DNS; Routing; Access Control List (ACL); Network Address Translation (NAT); Wireless Networking; Rule-Based Expert Systems; Large Language Models (LLM); Human-in-the-Loop AI; Responsible AI; Streamlit; Python; Pandas; Confidence Score; Root Cause Analysis.

List of Figures

Figure 4.1: Total System Architecture of NetSage AI System

Figure 4.2: Input, Process, Review & Output Layers View of the System

Figure 5.1: E2E Workflow of System from Case to Dashboard

Figure 6.1: Use Case Diagram – Network User, Human Reviewer & NetSage AI System

Figure 6.2: Data Flow Diagrams (Level 0 & Level 1)

Figure 6.3: Human Review Process (Accept/Modify/Reject)

Figure 6.4: Module Architecture Diagram

Figure 7.1: Project File Structure

Figure 9.1: Responsible AI Oversight Loop

List of Tables

Table 3.1 : Exigences Fonctionnelles

Table 3.2 : Exigences Non Fonctionnelles

Table 3.3 : System Constraints

Table 4.1: Technology Stack

Table 4.2: Advantages of the Proposed System

Table 5.1: Dataset Distribution by Networking Category

Table 5.2: OSI Layer Mapping

Table 6.1: cases.csv Schema

Table 6.2: human_review_log.csv Schema

Table 6.3: Module Description

Table 8.1: Test Case Summary

Table 8.2: Human Review Decision Categories

Table 10.1: Project Limitations

Table 10.2: Future Scope Roadmap

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
ACL	Access Control List
API	Application Programming Interface
CSV	Comma-Separated Values
DHCP	Dynamic Host Configuration Protocol
DNS	Domain Name System
IP	Internet Protocol
LAN	Local Area Network
LLM	Large Language Model
NAT	Network Address Translation
OSI	Open Systems Interconnection
RAG	Retrieval-Augmented Generation
SNMP	Simple Network Management Protocol
VLAN	Virtual Local Area Network
WLAN	Wireless Local Area Network
UI	User Interface
JSON	JavaScript Object Notation

Table of Contents

(Auto-generated — right-click and select 'Update Field' in Microsoft Word after opening to populate page numbers.)

Certificate / Approval Page	2
Student Declaration	3
Acknowledgement	4
Abstract	5-6
Keywords	7
List of Figures	8
List of Tables	9
List of Abbreviations	10
Table of Contents	11-12-13-14
Chapter 1: Introduction	15
1.1 Introduction	15
1.2 Background of Artificial Intelligence	15-16
1.3 Artificial Intelligence in Networking	16
1.4 Network Troubleshooting Challenges	16
1.5 Problem Statement	17
1.6 Need for the Proposed System	17
1.7 Project Objectives	17-18
1.8 Scope of the Project	18
1.9 Limitations	18
1.10 Organization of the Report	18-19
Chapter 2: Literature Review and Technical Background	20
2.1 Introduction	20
2.2 Traditional Network Troubleshooting	20
2.3 Cisco Network Troubleshooting Concepts	20-21
2.4 OSI Model in Troubleshooting	21
2.5 Rule-Based Expert Systems	21
2.6 Artificial Intelligence in Network Operations	21-22
2.7 Large Language Models for Technical Assistance	22
2.8 Human-in-the-Loop AI	22
2.9 Responsible AI	23
2.10 Research Gap	23
Chapter 3: Problem Analysis and Requirements	23
3.1 Existing Problem	24
3.2 Challenges Faced by Junior Network Engineers	24
3.3 Analysis of Network Fault Categories	24
3.4 Functional Requirements	24-25

3.5 Non-Functional Requirements.....	25
3.6 Dataset Requirements	25
3.7 Rule Checker Requirements.....	26
3.8 Diagnosis Engine Requirements	26
3.9 Human Review Requirements	26
3.10 Dashboard Requirements	26
3.11 System Constraints	26-27
Chapter 4: Proposed System.....	28
4.1 Introduction to NetSage AI	28
4.2 Proposed Solution.....	28
4.3 System Architecture.....	28
4.4 Input Layer	28-29
4.5 Dataset Layer	29
4.6 Python Rule Checker	29
4.7 AI-Oriented Diagnosis Layer	29-30
4.8 Mock Diagnosis Implementation.....	30
4.9 Human Review Layer	30
4.10 Review Logging.....	30
4.11 Dashboard	30-31
4.12 Responsible AI Architecture	31
4.13 Advantages of the Proposed System.....	31
Technology Stack (Table 4.1)	31-32
Chapter 5: Methodology.....	33
5.1 Development Methodology.....	33
5.2 Dataset Development Methodology.....	33
5.3 Case Categorization.....	33
Dataset Distribution (Table 5.1)	33-34
OSI Layer Mapping (Table 5.2).....	34
5.4 Rule-Based Validation Methodology.....	34
5.5 AI Diagnosis Methodology.....	34
5.6 Structured JSON Output	34-35
5.7 Human Review Methodology	35
5.8 Responsible AI Methodology.....	35
5.9 Testing Methodology	35
5.10 Evaluation Methodology	35-36
System Workflow — Mermaid Specification (Figure 5.1).....	36
Chapter 6: System Design	37
6.1 Overall System Architecture.....	37
6.2 Use Case Design	37
6.3 Use Case Diagram Description (Figure 6.1)	37

6.4 Data Flow Diagram (Figure 6.2)	37-38
6.5 Module Design	38
Module Description (Table 6.3)	38
6.6 Dataset Design	38
6.7 cases.csv Schema (Table — cases.csv).....	38-39
6.8 Human Review Log Schema (Table — human_review_log.csv)	39
6.9 Diagnosis JSON Schema (Table — diagnosis output).....	39
6.10 Human Review Workflow (Figure 6.3)	39-40
6.11 Module Architecture Diagram (Figure 6.4)	40
6.10 User Interface Design.....	40
6.11 Security Considerations	40
Chapter 7: System Implementation	41
7.1 Development Environment	41
7.2 Python Environment Setup.....	41
7.3 Library Installation.....	41
7.4 Project Folder Structure (Figure 7.1)	42
7.5 Dataset Implementation.....	42
7.6 Rule Checker Implementation	42-43
7.7 Diagnosis Engine Implementation.....	43
7.8 Prompt Engineering	43-44
7.9 Human Review Implementation.....	44
7.10 Review Logging Implementation	44
7.11 Streamlit Application Implementation.....	44
7.12 Dashboard Implementation	44
7.13 API Quota Challenge	45
7.14 Adaptation to Mock/Rule-Assisted Mode	45
Chapter 8: Testing and Results	46
8.1 Testing Strategy	46
8.2 Dataset Testing	46
8.3 Rule Checker Testing.....	46
8.4 Diagnosis Testing.....	46
8.5 Human Review Testing	46-47
8.6 Streamlit UI Testing.....	47
8.7 Example Test Cases (Table 8.1).....	47-48
8.8 Results and Discussion.....	48
8.9 System Strengths	48
8.10 System Limitations.....	48
Chapter 9: Responsible AI and Human Oversight.....	49
9.1 Introduction	49
9.2 Why Human Review is Necessary.....	49

9.3 Risks of AI Hallucination	49
9.4 Evidence-Based Diagnosis.....	49-50
9.5 Confidence Limitations	50
9.6 Accept/Edit/Reject Workflow	50
9.7 Human Review Logging.....	50
9.8 Example Corrected Diagnosis Cases	50
Demonstration Example 1	51
Demonstration Example 2	51
Demonstration Example 3	51
Demonstration Example 4	52
Demonstration Example 5	52
9.9 Responsible AI Benefits	52
9.10 Ethical Considerations	52-53
Chapter 10: Conclusion and Future Scope	54
10.1 Conclusion	54
10.2 Summary of Work	54
10.3 Project Achievements	54
10.4 Technical Contributions	54-55
10.5 Practical Contributions	55
10.6 Current Limitations	55
10.7 Future Scope	55-56
10.8 Final Remarks	56
References.....	57
Cisco Networking Academy Materials.....	57
Cisco Networking Documentation.....	57
Python Documentation.....	57
Streamlit Documentation	57
Pandas Documentation.....	57
OpenAI API Documentation.....	57
Responsible AI Documentation	57
Appendices.....	58
Appendix A: Sample Dataset Records	58
Appendix B: Troubleshooting Categories.....	58
Appendix C: Rule Checker Logic (Summary).....	58-59
Appendix D: Diagnosis JSON Structure.....	59
Appendix E: Human Review Log Structure.....	59
Appendix F: Sample Installation Instructions	59
Appendix G: Project Folder Structure.....	59
Appendix H: Screenshots to Include.....	59-60

Chapter 1: Introduction

1.1 Introduction

The development of computer networks is an essential element of all contemporary digital processes. The fast detection and solving of network problems is the basic skill that every network engineer needs. Cisco Networking Academy lessons and Packet Tracer laboratory activities are often employed as teaching material for learning such skills. Nonetheless, the transformation of theoretical protocol and command knowledge into a systematized approach to fault detection proves to be very challenging for students. This project, NetSage AI, was undertaken to explore how applied artificial intelligence techniques — combined with deterministic rule-based logic and mandatory human oversight — can support this learning and diagnostic process.

NetSage AI is seen as a tool that supports decisions and not a self-governing network administrator. It takes input regarding the network problem to be solved, with information on network topology, together with the output from the Cisco style show command. The NetSage tool then provides a structured and evidence-based diagnosis which is then to be reviewed by a human before it becomes official.

1.2 Background of Artificial Intelligence

Artificial Intelligence (AI) is the field of computer science that focuses on creating systems able to perform tasks requiring human intelligence, such as pattern recognition, natural language comprehension, and reasoning under uncertainty. Within the last decade, artificial intelligence moved from narrow rule-based expert systems through statistical machine learning to advanced large language models (LLMs) able to handle natural language processing and generation of structured technical information. Such progress made it possible to use AI not just for consumer applications, but also for specialized technical purposes like network management, where there is a great amount of log data, technical texts and diagnostics that could be analyzed with AI.

However, the increased capacity of AI was accompanied by increasing understanding of their drawbacks — specifically, their tendency to generate output that sounds plausible, but actually is not correct, which is called hallucination. This gave rise to human-in-the-loop

design pattern and other Responsible AI design patterns where output of the AI is used as recommendation and is subject to verification by humans. NetSage AI is deliberately designed around this philosophy.

1.3 Artificial Intelligence in Networking

Application of AI to networking includes the following: detecting anomalies in network traffic, predictive maintenance of networking hardware, configuration synthesis, and networking assistants using natural language processing techniques. AIOps, which is machine learning-based tool to correlate alerts and detect anomalies and provide possible remediation actions, is becoming more widespread. At the same time, large language models (LLMs) have been considered as a solution for networking conversational assistant that could interpret the output of show commands and give possible causes of the problem in networks.

However, most production-grade AI-for-networking tools are commercial, closed, and operate on live infrastructure telemetry, which makes them unsuitable as direct teaching tools for students working within a Cisco Networking Academy / Packet Tracer context. NetSage AI instead targets this educational and applied-AI niche: a transparent, inspectable system built from first principles using Python, in which both the deterministic rule logic and the intended AI reasoning process are made explicit and auditable.

1.4 Network Troubleshooting Challenges

Diagnosis of network problems involves solving a diagnostic problem because one symptom ("PC1 cannot connect to the file server") should be diagnosed backward through several layers of OSI model and configuration options. Junior engineers often possess command-level knowledge — they know how to run `show vlan brief` or `show ip route` — but lack a systematic method for narrowing down which command to run next, or for correctly interpreting the significance of the output once obtained.

The problem is exacerbated by the fact that one symptom can correspond to numerous potential sources of the problem at different layers of the OSI model, and some sources (such as VLAN configuration problems and gateway configuration problems) may generate almost identical symptoms. The lack of an evidence-based troubleshooting method can result in novice network administrators trying a trial-and-error approach to solving the problem.

1.5 Problem Statement

Develop and deploy a system to help solve network troubleshooting problems in a simulated lab environment similar to Cisco/Packet Tracer by allowing input of the network symptoms, topology information, and Cisco show command output which is analyzed through deterministic rule checks and an AI-oriented diagnosis engine; and produces a structured diagnosis comprising the likely fault, the relevant OSI layer, a next troubleshooting command, and an evidence-backed fix — subject to mandatory human review, correction, and logging, in line with Responsible AI principles.

1.6 Need for the Proposed System

- To help junior network engineers connect observed symptoms with likely root causes systematically rather than through ad hoc trial and error.
- To demonstrate how deterministic rule-based checks and AI-oriented reasoning can be combined rather than treated as competing approaches.
- To provide a transparent, evidence-based diagnosis format (root cause, OSI layer, confidence, evidence, next command, fix steps) that mirrors professional troubleshooting documentation.
- To embed human oversight as a mandatory step, addressing the well-documented risk of AI hallucination in technical domains.
- To create a reusable, extensible educational tool that can evolve from a mock diagnosis engine into a genuine LLM-backed assistant as resources permit.

1.7 Project Objectives

The specific objectives of this project are to:

1. Build an AI-assisted network troubleshooting system covering common Cisco fault categories.
2. Analyse network symptoms, topology notes, and Cisco-style show-command outputs as structured input.
3. Perform deterministic rule-based checking of configuration output to detect well-known fault signatures.
4. Identify likely root causes for a given troubleshooting case using a structured diagnosis engine.

5. Identify the relevant OSI layer associated with each diagnosed fault.
6. Provide supporting evidence and step-by-step remediation guidance for each diagnosis.
7. Include a mandatory human review step for every AI/mock diagnosis produced by the system.
8. Log all reviewer decisions (Accepted, Edited, Rejected) for future Responsible AI analysis.

1.8 Scope of the Project

The scope of NetSage AI, as implemented, covers a dataset of 36 troubleshooting cases spanning eight networking fault categories (VLAN, Default Gateway, DHCP, DNS, Routing, ACL, NAT, and Wireless Networking) and OSI Layers 1 through 7. The system includes a deterministic Python rule checker, a structured mock/rule-assisted diagnosis engine designed to mirror a future LLM-based engine, a human review workflow with Accepted/Edited/Rejected outcomes and logging, and a four-module Streamlit web application (Home, Diagnose Network, Human Review, and Dashboard). The scope explicitly excludes live integration with physical network devices, automated Packet Tracer topology ingestion, and production-grade database persistence, all of which are identified as future work.

1.9 Limitations

The current implementation has several limitations that are discussed in detail in Chapter 8 and Chapter 10, including: reliance on a mock/rule-assisted diagnosis engine rather than a live LLM API (due to an API quota constraint encountered during development), the use of a 36-case dataset based on structured/simulated scenarios rather than live network captures, CSV-based rather than database-backed storage, and conceptual rather than automated integration with Cisco Packet Tracer topologies. These limitations are stated transparently throughout this report and are not concealed.

1.10 Organization of the Report

This report is organised into ten chapters. Chapter 1 introduces the project. Chapter 2 reviews relevant technical background. Chapter 3 analyses the problem and derives requirements.

Chapter 4 presents the proposed system and its architecture. Chapter 5 describes the development methodology. Chapter 6 presents the detailed system design. Chapter 7 describes the implementation. Chapter 8 presents testing strategy and results. Chapter 9 discusses Responsible AI and human oversight in depth. Chapter 10 concludes the report and outlines future scope. References and appendices, including sample dataset records, diagram specifications, and screenshot placeholders, follow the main chapters.

Chapter 2: Literature Review and Technical Background

2.1 Introduction

This chapter situates NetSage AI within the broader technical landscape of network troubleshooting methodology, rule-based expert systems, and the emerging use of AI — particularly large language models — in technical support contexts. As instructed by the project brief, this discussion is conceptual and pedagogical in nature; no academic citations, authors, or DOIs have been fabricated. Locations where formal academic citations should be added following a literature search are explicitly marked as [CITATION NEEDED].

2.2 Traditional Network Troubleshooting

Traditional network troubleshooting methodologies, as taught in Cisco Networking Academy curricula, typically follow structured approaches such as the bottom-up method (starting at the Physical layer and working upward), the top-down method (starting at the Application layer and working downward), the divide-and-conquer method (starting at a middle layer, such as Network layer, and moving up or down based on findings), and the follow-the-path method (tracing the actual path of traffic through the network). [CITATION NEEDED — Cisco Networking Academy troubleshooting methodology reference.] These methodologies rely heavily on the engineer's own experience to select which show command to run next and how to interpret its output.

A common limitation of traditional troubleshooting, particularly for junior engineers, is that the choice of "next command" and the correct interpretation of its output are experience-driven rather than explicitly documented. NetSage AI attempts to encode a portion of this experiential knowledge as structured rules and, in its planned final form, as AI-generated recommendations validated by a human.

2.3 Cisco Network Troubleshooting Concepts

Cisco troubleshooting concepts commonly taught include VLAN configuration and trunking (including native VLAN mismatches and VLANs not permitted across a trunk), IP addressing and default gateway configuration, DHCP operation (including scope exhaustion and relay configuration), DNS resolution, static and dynamic routing, standard and extended ACL

behaviour (including the implicit deny at the end of every ACL), NAT/PAT translation, and wireless LAN configuration (SSID, security, and VLAN mapping). NetSage AI's dataset and rule checker are organised directly around these eight concept categories, reflecting their prominence in Cisco Networking Academy material and Packet Tracer laboratory exercises.

2.4 OSI Model in Troubleshooting

The seven-layer OSI reference model — Physical, Data Link, Network, Transport, Session, Presentation, and Application — provides a structured vocabulary for describing where in the communication stack a fault occurs. In practice, Cisco troubleshooting exercises most frequently reference Layer 1 (physical/interface status), Layer 2 (VLAN/switching), Layer 3 (IP addressing, routing, NAT), Layer 4 (transport-level filtering), and Layer 7 (application-level services such as DNS and DHCP). NetSage AI explicitly tags every case and every diagnosis with its associated OSI layer so that the reasoning process mirrors this widely taught framework, reinforcing rather than replacing the learner's conceptual model.

2.5 Rule-Based Expert Systems

Rule-based expert systems represent one of the earliest successful applications of AI to technical diagnosis. Such systems encode expert knowledge as a set of if-then rules (for example, "if the show output contains the phrase 'administratively down', then flag a Layer 1 interface issue") and apply these rules deterministically to input data. Their key strengths are transparency, predictability, and ease of validation, since every conclusion can be traced back to the specific rule that produced it. Their key weakness is limited coverage: a rule-based system can only diagnose faults for which a rule has been explicitly authored, and it cannot generalise to novel or compound fault patterns the way a statistical or generative model might. [CITATION NEEDED — expert systems literature.]

NetSage AI's Python rule checker is a direct application of this rule-based expert-system paradigm, covering eleven specific configuration anti-patterns as described in Chapter 4 and Chapter 7.

2.6 Artificial Intelligence in Network Operations

Beyond rule-based systems, statistical and machine-learning approaches have been applied to network operations for tasks such as traffic anomaly detection, capacity forecasting, and root-cause correlation across large volumes of alerts. [CITATION NEEDED — AIOps / network AI literature.] More recently, generative AI and large language models have been proposed as natural-language interfaces to network diagnosis, capable of interpreting free-text symptom descriptions and semi-structured show-command output and producing human-readable explanations and recommendations. NetSage AI's diagnosis-engine architecture (Chapter 4, Chapter 7) was designed with this LLM-based approach in mind, even though the present implementation substitutes a mock/rule-assisted engine due to the API quota limitation described in Section 7.13.

2.7 Large Language Models for Technical Assistance

Large language models are trained on large corpora of text, including technical documentation, and can generate contextually relevant natural-language output given a prompt. When applied to technical assistance, LLMs are typically constrained through prompt engineering to (a) reason only over the evidence explicitly supplied to them, (b) avoid inventing configuration details not present in the input, and (c) produce output in a structured, machine-parseable format (such as JSON) to support downstream processing. Section 6.9 and Appendix D of this report present the structured JSON schema designed for NetSage AI's diagnosis output, which was authored specifically to support this style of constrained, evidence-grounded LLM prompting once a live API becomes available.

2.8 Human-in-the-Loop AI

Human-in-the-loop (HITL) AI refers to system designs in which a human reviewer remains part of the decision-making loop, either by validating AI output before it is acted upon, or by providing corrective feedback that can be used to improve the system over time. HITL design is particularly important in domains where incorrect AI output could have significant consequences — including, in the networking context, a junior engineer acting on an incorrect diagnosis and causing further outages. NetSage AI implements HITL directly through its mandatory Accepted/Edited/Rejected human review workflow, described in detail in Chapter 6 and Chapter 9.

2.9 Responsible AI

Responsible AI is an umbrella term for practices intended to ensure that AI systems are used safely, transparently, and accountably. Commonly cited Responsible AI principles include transparency (making it clear how and why a system produced a given output), human oversight (ensuring a human can review, override, or reject AI decisions), evidence-based reasoning (grounding conclusions in verifiable input rather than unconstrained generation), and honest reporting of system limitations. NetSage AI was explicitly designed to embody these principles: every diagnosis carries supporting evidence, every diagnosis requires human review, and this report itself follows a policy of transparently disclosing where the AI component is currently simulated rather than live.

2.10 Research Gap

While rule-based network diagnostic tools and, separately, general-purpose LLM assistants both exist in various forms, there is comparatively little publicly documented work — particularly at the student/applied-AI-internship level — that combines (a) a structured, Cisco-curriculum-aligned troubleshooting dataset, (b) a deterministic rule checker, (c) an AI-oriented diagnosis engine designed for constrained, evidence-grounded LLM prompting, and (d) a mandatory, logged human review workflow, into a single cohesive, transparently documented educational system. NetSage AI aims to address this gap at a scale and depth appropriate to an academic/internship project, while remaining honest about the current substitution of a mock diagnosis engine for a live LLM backend.

Chapter 3: Problem Analysis and Requirements

3.1 Existing Problem

As introduced in Chapter 1, junior network engineers frequently possess command-level knowledge of Cisco IOS but lack a systematic method for mapping an observed symptom to its root cause across the eight fault categories addressed by this project. Existing Cisco Networking Academy materials teach troubleshooting methodology conceptually, but there is no lightweight, inspectable, AI-assisted tool that a learner can use interactively against a structured set of practice cases while receiving evidence-based, OSI-layer-tagged diagnostic feedback, subject to human validation.

3.2 Challenges Faced by Junior Network Engineers

- Difficulty selecting the correct next diagnostic command when a symptom could originate from multiple OSI layers.
- Difficulty correctly interpreting show-command output (for example, recognising that 'administratively down' indicates a manually disabled interface rather than a cabling fault).
- Limited exposure to the range of fault categories (VLAN, gateway, DHCP, DNS, routing, ACL, NAT, wireless) that can produce superficially similar symptoms.
- Lack of structured feedback distinguishing a correct diagnosis from an incorrect one during self-study.
- Risk of over-reliance on unverified AI suggestions if no human-review discipline is established early in their training.

3.3 Analysis of Network Fault Categories

The eight fault categories addressed by NetSage AI were selected because they collectively span the OSI layers most commonly exercised in Cisco Networking Academy and Packet Tracer laboratory exercises, and because each category has well-known, teachable configuration anti-patterns that can be reliably detected from show-command text. Table 3.1 through Table 3.3 (later in this chapter and in Chapter 5) summarise the requirements and dataset distribution derived from this analysis.

3.4 Functional Requirements

ID	Functional Requirement
FR-1	The system shall allow a user to select a troubleshooting case from the dataset.
FR-2	The system shall display the case symptom, topology note, and show-command output.
FR-3	The system shall run a deterministic rule checker against the show-command output.
FR-4	The system shall generate a structured diagnosis (root cause, OSI layer, confidence, evidence, next command, fix steps).
FR-5	The system shall require a human reviewer to evaluate every diagnosis before it is considered final.
FR-6	The system shall allow the reviewer to mark a diagnosis as Accepted, Edited, or Rejected.
FR-7	The system shall allow the reviewer to enter a corrected final diagnosis and reviewer notes.
FR-8	The system shall log every review decision with a timestamp and case identifier.
FR-9	The system shall display dashboard statistics of the dataset (totals, by concept, by severity).
FR-10	The system shall be operable entirely through a Streamlit web interface.

3.5 Non-Functional Requirements

ID	Non-Functional Requirement
NFR-1	Usability — the interface shall be understandable to a student with basic networking knowledge.
NFR-2	Transparency — every diagnosis shall be accompanied by explicit supporting evidence.
NFR-3	Auditability — every human review decision shall be persistently logged.
NFR-4	Extensibility — the diagnosis engine architecture shall support future substitution of a live LLM API without redesign.
NFR-5	Portability — the system shall run within a standard Python virtual environment without external infrastructure.
NFR-6	Maintainability — the rule checker and dataset shall be stored as plain text/CSV for easy inspection and extension.
NFR-7	Honesty — the system and its documentation shall not misrepresent mock output as live AI output.

3.6 Dataset Requirements

The dataset must contain at least 30 troubleshooting cases (36 were implemented) covering the eight fault categories, with each case specifying a case identifier, symptom, topology note, show-command output, expected fault, OSI layer, concept, and severity, as detailed in Section 6.7.

3.7 Rule Checker Requirements

The rule checker must deterministically scan show-command output text for at least the eleven configuration anti-patterns listed in Section 4.6 and Section 7.6, returning structured findings comprising an issue description, severity, evidence, and recommendation for each match.

3.8 Diagnosis Engine Requirements

The diagnosis engine must accept the symptom, topology note, show-command output, and rule-checker findings as input, and must produce output conforming to the structured JSON schema defined in Section 6.9, including a `requires_human_review` flag that is always set to `true`.

3.9 Human Review Requirements

The human review module must present the diagnosis and its evidence to the reviewer, accept one of three decisions (Accepted, Edited, Rejected), allow entry of a final diagnosis and reviewer notes, and persist this information to the human review log with a timestamp and case identifier.

3.10 Dashboard Requirements

The dashboard must display, at minimum, the total number of cases, a breakdown of cases by networking concept, a breakdown of cases by severity, and general dataset information, with future extensibility toward acceptance/edit/rejection-rate analytics as outlined in Section 4.11 and Section 10.7.

3.11 System Constraints

Constraint	Description
API Quota	Live OpenAI-compatible API calls were not available due to an HTTP 429 <code>insufficient_quota</code> error; a mock/rule-assisted engine was substituted.
Storage	CSV files are used for the dataset and review log instead of a production database.
Topology Integration	Packet Tracer topology import is conceptual/manual rather than automated.

Constraint	Description
Scale	The dataset (36 cases) is designed for demonstration and educational use rather than production-scale coverage.
Environment	The system is designed to run within a local Python virtual environment rather than a hosted/cloud deployment.

Chapter 4: Proposed System

4.1 Introduction to NetSage AI

NetSage AI is proposed as a hybrid AI-assisted and deterministic rule-based network troubleshooting system with mandatory human review. It is designed to take a Cisco-style troubleshooting case as input and produce a structured, evidence-based diagnosis that a human reviewer validates before it is treated as authoritative. The system is intentionally modular, separating dataset management, deterministic rule checking, diagnosis generation, human review, and presentation into distinct layers.

4.2 Proposed Solution

The proposed solution combines three complementary techniques: (1) a deterministic rule checker that reliably flags well-known Cisco configuration anti-patterns directly from show-command text; (2) a structured diagnosis engine, architected for a future LLM backend but currently implemented as a mock/rule-assisted engine, that synthesises the symptom, topology note, and rule findings into a single coherent diagnosis; and (3) a mandatory human review layer that captures a reviewer's judgment on every diagnosis and logs it for future Responsible AI analysis. Together, these layers ensure that the system's output is both technically grounded and subject to human accountability.

4.3 System Architecture

The system architecture consists of five layers: the Input Layer (case selection and display), the Dataset Layer (the CSV-based case repository), the Rule-Checking Layer (deterministic Python logic), the Diagnosis Layer (mock/rule-assisted engine with a schema compatible with a future LLM), and the Human Review and Presentation Layer (Streamlit-based review workflow, logging, and dashboard). Figure 4.1 (see Appendix H for diagram placeholder and Chapter 6 for the accompanying Mermaid specification) illustrates how these layers interact.

4.4 Input Layer

The Input Layer is responsible for presenting the user with a selectable list of troubleshooting cases and displaying the associated symptom, topology note, and show-command output

once a case is selected. This layer is implemented within the Diagnose Network module of the Streamlit application (Section 7.11).

4.5 Dataset Layer

The Dataset Layer consists of the `data/cases.csv` file, which stores all 36 troubleshooting cases with the schema described in Section 6.7. This layer is read using Pandas at application start-up and provides the data backing both the Diagnose Network module and the Dashboard module.

4.6 Python Rule Checker

The rule-checking layer, implemented in `src/rule_checker.py`, applies deterministic pattern checks to the show-command output associated with a case. It is designed to detect, at minimum, the following eleven configuration anti-patterns:

9. Interface administratively down
10. Missing VLAN
11. VLAN not allowed on trunk
12. Missing default gateway
13. Gateway mismatch
14. Missing route
15. Incorrect next-hop
16. DHCP pool exhaustion
17. Missing DHCP relay
18. ACL implicit deny
19. Overly permissive ACL

Each detected issue is returned as a structured finding containing an issue description, severity, evidence (the specific phrase or condition matched in the show output), and a recommendation, as illustrated in Section 7.6.

4.7 AI-Oriented Diagnosis Layer

The diagnosis layer was architected to accept the symptom, topology note, show-command output, and rule-checker findings, and to return a structured JSON object containing

`root_cause`, `osi_layer`, `confidence`, `evidence`, `next_command`, `fix_steps`, and `requires_human_review`, as specified in Section 6.9. This layer was originally intended to call an OpenAI-compatible LLM API to perform this synthesis using natural-language reasoning constrained by the structured prompt described in Section 7.8.

4.8 Mock Diagnosis Implementation

Because the live API call returned an HTTP 429 `insufficient_quota` error during development and testing (Section 7.13), the diagnosis layer was implemented as a mock/rule-assisted engine (`src/ai_diagnosis.py`). This engine derives its output directly and deterministically from the rule-checker findings and case metadata rather than from a live generative model, while still conforming exactly to the structured JSON schema that a future LLM-based implementation would need to satisfy. This design choice ensures that the remainder of the system — human review, logging, and dashboard — can be developed, demonstrated, and tested end-to-end without depending on a paid, quota-limited external API.

4.9 Human Review Layer

The Human Review layer, implemented in `src/reviewer.py` and the Human Review Streamlit module, presents the generated diagnosis and its evidence to a human reviewer, who selects Accepted, Edited, or Rejected, optionally supplies a corrected final diagnosis and notes, and submits the review. No diagnosis is treated as final until this step is completed.

4.10 Review Logging

Every review decision is appended to `data/human_review_log.csv`, capturing the timestamp, case identifier, the original AI/mock diagnosis, the human decision, the final diagnosis, and reviewer notes, as detailed in Section 6.8. This log forms the basis for the Responsible AI analysis discussed in Chapter 9 and for the future analytics discussed in Section 4.11 and Section 10.7.

4.11 Dashboard

The Dashboard module presents summary statistics drawn from the dataset, including total case count, distribution by networking concept, and distribution by severity. Because the human review log accumulates decision data over time, the dashboard is designed to be

extended in future work to display acceptance rate, edit rate, rejection rate, AI-human agreement, and confidence-versus-correctness analysis, as outlined in Section 10.7.

4.12 Responsible AI Architecture

Responsible AI principles are embedded structurally rather than added as an afterthought: every diagnosis must carry evidence; every diagnosis must be reviewed by a human; every review decision is logged; and the system's documentation (including this report) transparently discloses that the current diagnosis engine is a mock/rule-assisted implementation rather than a live LLM, pending resolution of the API quota constraint described in Section 7.13.

4.13 Advantages of the Proposed System

Advantage	Description
Transparency	Every diagnosis is accompanied by explicit, traceable evidence rather than an opaque conclusion.
Human Accountability	No diagnosis is auto-applied; a human reviewer must accept, edit, or reject it.
Extensibility	The mock diagnosis engine's JSON schema is designed to be a drop-in replacement point for a live LLM.
Educational Value	Encourages learners to reason through OSI layers and evidence rather than guess-and-check.
Low Infrastructure Cost	Runs on CSV storage and a local Python environment without external dependencies for its core logic.
Auditability	All review decisions are permanently logged for later Responsible AI analysis.

Technology Stack (Table 4.1)

Component	Technology Used
Programming Language	Python
Web Framework	Streamlit
Data Processing	Pandas
Visualization	Streamlit native charts; optionally Plotly
Environment Management	Python virtual environment (venv)
Data Storage	CSV files (cases.csv, human_review_log.csv)
AI Architecture (Planned)	OpenAI-compatible API architecture
AI Implementation (Current)	Mock/rule-assisted diagnosis engine

Component	Technology Used
Development Environment	Visual Studio Code
Networking Context	Cisco networking concepts, Packet Tracer scenarios, Cisco-style show-command output

Chapter 5: Methodology

5.1 Development Methodology

This project followed an iterative, prototype-based development methodology rather than a rigid waterfall process. Each subsystem — dataset, rule checker, diagnosis engine, human review, and Streamlit interface — was developed as an initial working prototype, tested informally, and then refined based on observed gaps (most notably, the pivot from a live-API diagnosis engine to a mock/rule-assisted engine after the API quota limitation was encountered). This approach was chosen because it allowed the overall workflow to be demonstrated end-to-end early, with individual components strengthened incrementally.

5.2 Dataset Development Methodology

The dataset was developed by first enumerating the eight target fault categories, then authoring representative cases for each category drawing on common Cisco Networking Academy and Packet Tracer troubleshooting scenarios. For each case, a realistic symptom description, a topology note, simplified Cisco-style show-command output, and metadata (expected fault, OSI layer, concept, severity) were authored consistently so that the dataset could be consumed uniformly by both the rule checker and the diagnosis engine.

5.3 Case Categorization

Cases were categorised into eight networking concepts — VLAN, Default Gateway, DHCP, DNS, Routing, ACL, NAT, and Wireless Networking — with a target of four to five cases per category, as shown in Table 5.1, to ensure the dataset covers the required OSI layers with reasonable balance rather than concentrating on a single fault type.

Dataset Distribution (Table 5.1)

Networking Category	Number of Cases
VLAN	5
Default Gateway	4
DHCP	5
DNS	4
Routing	5

Networking Category	Number of Cases
ACL	5
NAT	4
Wireless	4
Total	36

OSI Layer Mapping (Table 5.2)

OSI Layer	Typical Fault Domains Covered
Layer 1 — Physical	Interface/physical status problems (e.g., administratively down).
Layer 2 — Data Link	VLAN configuration, switching, trunking, wireless VLAN mapping.
Layer 3 — Network	IP addressing, default gateway, routing, NAT, network-level ACLs.
Layer 4 — Transport	Transport-level traffic filtering where applicable.
Layer 7 — Application	DHCP, DNS, authentication, and other application-level services.

5.4 Rule-Based Validation Methodology

The rule checker was developed by identifying, for each of the eleven target configuration anti-patterns (Section 4.6), a characteristic textual signature that would appear in simplified Cisco-style show-command output (for example, the literal phrase 'administratively down', or the absence of a VLAN entry expected by the topology note). Each rule was implemented as an independent, testable Python function returning a structured finding, allowing the rule checker to be validated rule-by-rule using the unit tests described in Section 8.3.

5.5 AI Diagnosis Methodology

The diagnosis engine's methodology was designed around a two-stage plan: first, define the structured input/output contract (Section 6.9) that any diagnosis engine implementation — live LLM or mock — must satisfy; second, implement a first version of the engine against that contract. Because the live LLM stage could not be completed due to the API quota constraint, the mock/rule-assisted implementation directly derives its `root_cause`, `osi_layer`, `evidence`, `next_command`, and `fix_steps` fields from the rule-checker findings and case metadata, ensuring the output remains evidence-grounded even without a generative model.

5.6 Structured JSON Output

A structured JSON output format was adopted from the outset (rather than free-text output) to ensure the diagnosis engine's output would be machine-parseable by the Streamlit interface regardless of whether it originated from a mock implementation or a future live LLM. This schema is presented in full in Section 6.9 and Appendix D.

5.7 Human Review Methodology

The human review workflow was designed around three explicit outcomes — Accepted, Edited, Rejected — chosen because they map cleanly onto the three possible relationships between an AI/mock diagnosis and ground truth: fully correct, partially correct, and incorrect. This three-way outcome also supports the acceptance-rate, edit-rate, and rejection-rate analytics proposed in Section 4.11 and Section 10.7.

5.8 Responsible AI Methodology

Responsible AI considerations were incorporated methodologically by (a) requiring every diagnosis-generation code path to populate an evidence field, (b) hard-coding `requires_human_review` as true in the diagnosis schema so it cannot be bypassed, and (c) ensuring the review log schema captures both the original diagnosis and the human's final judgment, preserving a complete audit trail rather than overwriting the AI's original output.

5.9 Testing Methodology

Testing was carried out primarily through targeted unit tests (`src/test_dataset.py`, `src/test_rule_checker.py`, `src/test_reviewer.py`) covering dataset integrity, rule-checker correctness against known show-output signatures, and reviewer logging behaviour, supplemented by manual, demonstration-style testing of the Streamlit interface. Full details and results are presented in Chapter 8.

5.10 Evaluation Methodology

Because the diagnosis engine currently operates in mock/rule-assisted mode rather than against a live LLM, formal AI-accuracy evaluation metrics (such as precision/recall of root-cause identification against held-out cases) are proposed as future evaluation rather than reported as completed results, consistent with the instruction not to fabricate performance

metrics. Section 8.8 and Section 10.7 describe the proposed evaluation methodology to be applied once live LLM integration is completed.

System Workflow — Mermaid Specification (Figure 5.1)

The following Mermaid flowchart specifies the end-to-end system workflow described narratively in the project background and implemented across the modules discussed in Chapter 4, Chapter 6, and Chapter 7. It should be rendered as Figure 5.1 in the final formatted report.

```

flowchart TD
    A[Network Case] --> B[Network Symptom]
    B --> C[Topology Information]
    C --> D[Cisco-Style Show Output]
    D --> E[Python Deterministic Rule Checker]
    E --> F[Rule-Based Findings]
    F --> G[AI / Mock Diagnosis Engine]
    G --> H[Structured Diagnosis]
    H --> I[Mandatory Human Review]
    I --> J{Accepted / Edited / Rejected}
    J --> K[Human Review Log]
    K --> L[Dashboard and Analytics]
  
```

Figure 5.1: End-to-End System Workflow (Case to Dashboard)

Chapter 6: System Design

6.1 Overall System Architecture

The overall system architecture, introduced in Chapter 4, is realised in this chapter through detailed design artefacts: a use case diagram identifying system actors and their interactions, a data flow diagram describing information movement through the system, a module design showing the five core Python/Streamlit modules, and full schema definitions for the dataset, review log, and diagnosis JSON output.

6.2 Use Case Design

Three actors interact with NetSage AI:

- Network User / Engineer — selects a troubleshooting case, views symptoms and evidence, and initiates diagnosis generation.
- Human Reviewer — reviews each generated diagnosis and records an Accepted, Edited, or Rejected decision with optional corrections and notes.
- NetSage AI System — executes the rule checker and mock/rule-assisted diagnosis engine, and persists review outcomes to the log.

6.3 Use Case Diagram Description (Figure 6.1)

```

flowchart LR
    U([Network User / Engineer]) --> UC1[Select Troubleshooting Case]
    U --> UC2[View Symptom & Evidence]
    U --> UC3[Run Diagnosis]
    R([Human Reviewer]) --> UC4[Review AI/Mock Diagnosis]
    R --> UC5[Accept / Edit / Reject]
    R --> UC6[Add Reviewer Notes]
    S([NetSage AI System]) --> UC7[Run Rule Checker]
    S --> UC8[Generate Structured Diagnosis]
    S --> UC9[Log Review Decision]
    S --> UC10[Render Dashboard]
  
```

Figure 6.1: Use Case Diagram — Network User, Human Reviewer, NetSage AI System

6.4 Data Flow Diagram (Figure 6.2)

```

flowchart TD
    subgraph Level0 [Level 0 - Context]
        U1[User] -->|Case Selection| SYS0((NetSage AI))
        SYS0 -->|Diagnosis + Review UI| U1
    end
  
```

```

subgraph Level1[Level 1 - Detailed]
  DS[(cases.csv)] --> P1[Rule Checker]
  P1 --> P2[Diagnosis Engine]
  P2 --> P3[Human Review]
  P3 --> LOG[(human_review_log.csv)]
  DS --> P4[Dashboard Aggregation]
  LOG --> P4
end

```

Figure 6.2: Data Flow Diagram (Level 0 and Level 1)

6.5 Module Design

The system is composed of five core modules, described in Table 6.3, corresponding directly to the files under `src/` and the Streamlit pages within `app.py`.

Module Description (Table 6.3)

Module	File	Responsibility
Dataset Module	<code>data/cases.csv</code>	Stores the 36 troubleshooting cases and their metadata.
Rule Checker Module	<code>src/rule_checker.py</code>	Detects deterministic configuration anti-patterns from show output.
Diagnosis Module	<code>src/ai_diagnosis.py</code>	Synthesises a structured diagnosis (currently mock/rule-assisted).
Human Review Module	<code>src/reviewer.py</code>	Captures and logs human review decisions.
Dashboard Module	<code>app.py</code> (Dashboard page)	Aggregates and displays dataset statistics.

6.6 Dataset Design

The dataset is designed as a flat CSV file to maximise transparency and ease of inspection, in keeping with the project's emphasis on auditability. Each row represents one self-contained troubleshooting case.

6.7 cases.csv Schema (Table — cases.csv)

Field	Description
<code>case_id</code>	Unique identifier for the case (e.g., CASE001).
<code>symptom</code>	Free-text description of the observed network problem.
<code>topology_note</code>	Free-text description of the intended network design.
<code>show_output</code>	Simplified Cisco-style show-command output relevant to the case.
<code>expected_fault</code>	The ground-truth fault used for dataset design and future evaluation.

Field	Description
osi_layer	The OSI layer associated with the fault (e.g., Layer 2).
concept	The networking concept/category (e.g., VLAN, DHCP).
severity	Severity label (e.g., High, Medium, Low).

6.8 Human Review Log Schema (Table — human_review_log.csv)

Field	Description
timestamp	Date and time the review was submitted.
case_id	Identifier of the case being reviewed.
ai_diagnosis	The original diagnosis produced by the mock/AI engine.
human_decision	One of: Accepted, Edited, Rejected.
final_diagnosis	The reviewer-confirmed or reviewer-corrected diagnosis.
reviewer_notes	Free-text notes explaining the reviewer's decision.

6.9 Diagnosis JSON Schema (Table — diagnosis output)

The diagnosis engine (mock implementation, and any future live-LLM implementation) is designed to conform to the following structured JSON schema:

```
{
  "root_cause": "",
  "osi_layer": "",
  "confidence": 0,
  "evidence": [],
  "next_command": "",
  "fix_steps": [],
  "requires_human_review": true
}
```

The `requires_human_review` field is fixed to `true` in the current implementation, ensuring that no code path can bypass the mandatory human review process described in Section 6.2 and Chapter 9.

6.10 Human Review Workflow (Figure 6.3)

```
flowchart LR
    A[Diagnosis Generated] --> B{Human Reviewer Evaluates}
    B -->|Correct| C[Accepted]
    B -->|Partially Correct| D[Edited + Final Diagnosis]
    B -->|Incorrect| E[Rejected + Notes]
    C --> F[Logged to human_review_log.csv]
    D --> F
    E --> F
```



```

graph LR
    E --> F
  
```

Figure 6.3: Human Review Workflow (Accept / Edit / Reject)

6.11 Module Architecture Diagram (Figure 6.4)

```

flowchart TB
    App["App[app.py - Streamlit Entry Point]"] --> Home["Home[Home Page]"]
    App --> Diagnose["Diagnose[Diagnose Network Page]"]
    App --> Review["Review[Human Review Page]"]
    App --> Dashboard["Dashboard[Dashboard Page]"]
    Diagnose --> RC["RC[rule_checker.py]"]
    Diagnose --> AI["AI[ai_diagnosis.py]"]
    Review --> Rev["Rev[reviewer.py]"]
    Rev --> Log["Log[(human_review_log.csv)]"]
    Dashboard --> Data["Data[(cases.csv)]"]
  
```

Figure 6.4: Module Architecture Diagram

6.10 User Interface Design

The user interface is implemented entirely in Streamlit, using a page-based navigation structure comprising Home, Diagnose Network, Human Review, and Dashboard pages, as described in Section 7.11. Interface design favours clarity and evidence visibility over visual complexity, consistent with the system's educational purpose.

6.11 Security Considerations

- API keys (for the originally planned live LLM integration) are stored in a local `.env` file rather than hard-coded into source files.
- The `.env` file is excluded from version control via `.gitignore` to prevent accidental credential exposure.
- Because every diagnosis requires mandatory human review before being acted upon, the system's human-in-the-loop design mitigates the risk of blind automation of potentially incorrect network changes.
- As the system currently uses local CSV storage without authentication, it is intended for single-user/local educational use rather than multi-user production deployment; user authentication and role-based access are identified as future work in Section 10.7.

Chapter 7: System Implementation

7.1 Development Environment

The system was developed using Visual Studio Code as the primary integrated development environment, running on a standard desktop/laptop operating system with Python 3 installed. Version control practices included the use of a .gitignore file to exclude the virtual environment directory and environment-variable files from source control.

7.2 Python Environment Setup

A dedicated Python virtual environment (venv) was created for the project to isolate its dependencies from the system-wide Python installation. This ensures that the specific versions of Streamlit, Pandas, and other libraries used by NetSage AI do not conflict with other Python projects on the development machine.

```
python -m venv venv
venv\Scripts\activate      (Windows)
source venv/bin/activate    (macOS/Linux)
pip install -r requirements.txt
```

7.3 Library Installation

The following core libraries were installed via requirements.txt:

Library	Purpose
streamlit	Web application framework used to build the four-page user interface.
pandas	Reading, filtering, and aggregating the CSV-based dataset and review log.
plotly	Optional richer charting for the Dashboard module.
python-dotenv	Loading environment variables (including the originally planned API key) from a local .env file.
openai	Client library installed for the originally planned live LLM API integration.

The openai library was installed and the corresponding client-initialisation code was written as part of the originally planned architecture. However, as described in Section 7.13, live calls through this library returned a quota-related error, and the diagnosis engine was consequently adapted to the mock/rule-assisted implementation described in Section 7.14, while the library and its integration point remain in the codebase for future use.

7.4 Project Folder Structure (Figure 7.1)



Figure 7.1: Project Folder Structure

7.5 Dataset Implementation

The dataset was implemented as `data/cases.csv`, with each row representing one of the 36 troubleshooting cases and columns matching the schema in Section 6.7. An illustrative record is reproduced below.

Field	Example Value (CASE001)
case_id	CASE001
symptom	PC1 cannot communicate with PC2 in the same department.
topology_note	Both PCs should be in VLAN 10.
show_output	show vlan brief: VLAN 10 missing
expected_fault	Missing VLAN
osi_layer	Layer 2
concept	VLAN
severity	High

7.6 Rule Checker Implementation

The rule checker inspects the `show_output` text associated with a case for known fault signatures. A representative, simplified excerpt illustrating the pattern used for the 'interface administratively down' check is shown below; the actual implementation applies a similar structured-finding pattern across all eleven checks listed in Section 4.6.

```
def check_admin_down(show_output):
    if 'administratively down' in show_output.lower():
        return {
            'issue': 'Interface is administratively down',
            'severity': 'High',
            'evidence': "show output contains 'administratively down'",
            'recommendation': "Check the interface and use 'no shutdown' if
appropriate."
        }
    return None
```

7.7 Diagnosis Engine Implementation

The diagnosis engine (`src/ai_diagnosis.py`) receives the symptom, topology note, show output, and rule-checker findings, and produces a dictionary conforming to the schema in Section 6.9. In the current mock/rule-assisted implementation, the root cause, OSI layer, and evidence fields are derived directly from the highest-severity rule-checker finding and the case's concept metadata, rather than from a live generative model.

```
def generate_diagnosis(symptom, topology_note, show_output, findings):
    top_finding = pick_highest_severity(findings)
    return {
        'root_cause': top_finding['issue'] if top_finding else 'Undetermined',
        'osi_layer': infer_osi_layer(top_finding),
        'confidence': estimate_confidence(top_finding),
        'evidence': [top_finding['evidence']] if top_finding else [],
        'next_command': suggest_next_command(top_finding),
        'fix_steps': [top_finding['recommendation']] if top_finding else [],
        'requires_human_review': True
    }
```

7.8 Prompt Engineering

A structured prompt was authored in `prompts/diagnose_prompt.md` for the originally planned live LLM integration. The prompt instructs the model to reason strictly from supplied evidence, avoid inventing configuration details, and return output in the exact JSON schema defined in Section 6.9. This prompt is retained in the codebase and is intended to be used

unchanged once live API access is restored, ensuring continuity between the current mock implementation and any future LLM-backed version.

```
You are a Cisco network troubleshooting assistant.  
Use ONLY the symptom, topology note, show output, and rule findings provided.  
Do NOT invent configuration details not present in the input.  
Identify the likely root cause and the relevant OSI layer.  
Provide a confidence score (0-100) and list the supporting evidence.  
Suggest the next troubleshooting command and step-by-step fix steps.  
Always set requires_human_review to true.  
Return your answer strictly as JSON matching the required schema.
```

7.9 Human Review Implementation

The Human Review module (`src/reviewer.py`) presents the generated diagnosis and evidence to the user through Streamlit widgets, captures the selected decision (Accepted, Edited, Rejected) along with any corrected final diagnosis and reviewer notes, and passes this information to the logging function described in Section 7.10.

7.10 Review Logging Implementation

Review submissions are appended as new rows to `data/human_review_log.csv` using Pandas, preserving the full history of every review decision made during use of the system, in line with the schema defined in Section 6.8.

7.11 Streamlit Application Implementation

The Streamlit application (`app.py`) is organised into four pages: Home (project overview and workflow explanation), Diagnose Network (case selection, rule checking, and diagnosis generation), Human Review (review decision capture), and Dashboard (dataset statistics). Navigation between pages is handled through Streamlit's standard multi-page or sidebar-based navigation mechanism.

7.12 Dashboard Implementation

The Dashboard page loads `data/cases.csv` via Pandas and computes aggregate counts by concept and severity, rendering them using Streamlit's built-in charting functions (and optionally Plotly), consistent with the requirements in Section 3.10.

7.13 API Quota Challenge

During development, the diagnosis engine's originally planned call to an OpenAI-compatible API returned an HTTP 429 RateLimitError with the reason `insufficient_quota`. This indicated that the associated API account/key did not have sufficient quota to service live requests, preventing a live demonstration of LLM-based diagnosis generation within the scope and budget of this academic/internship project.

7.14 Adaptation to Mock/Rule-Assisted Mode

In response to this constraint, the project was adapted so that `ai_diagnosis.py` implements a mock/rule-assisted diagnosis engine that satisfies the same input/output contract as the originally planned LLM-based engine (Section 6.9), allowing the complete workflow — rule checking, diagnosis, human review, logging, and dashboard analytics — to be developed, demonstrated, and tested end-to-end. The prompt engineering artefact from Section 7.8 and the `openai` client dependency remain in place so that live LLM integration can be substituted into the same architecture in future work (Section 10.7), without requiring redesign of the surrounding system.

Chapter 8: Testing and Results

8.1 Testing Strategy

Testing of NetSage AI combined automated unit tests for deterministic components (dataset integrity and rule-checker logic) with manual, demonstration-style testing of components that depend on the Streamlit interface and the mock diagnosis engine. Because the diagnosis engine currently operates without a live LLM backend, tests involving diagnosis generation are reported as outcomes of the implemented rule-assisted logic, and any scenario that would require a live model is explicitly labelled as a proposed or demonstration test rather than a completed live-AI result.

8.2 Dataset Testing

`src/test_dataset.py` verifies that `data/cases.csv` contains the expected number of records, that all required columns (`case_id`, `symptom`, `topology_note`, `show_output`, `expected_fault`, `osi_layer`, `concept`, `severity`) are present, and that case identifiers are unique.

8.3 Rule Checker Testing

`src/test_rule_checker.py` exercises each of the eleven rule-checker functions against representative show-output strings, verifying that the correct issue, severity, evidence, and recommendation are returned for both matching and non-matching input.

8.4 Diagnosis Testing

Diagnosis testing verifies that, given a set of rule-checker findings, the mock diagnosis engine returns a well-formed dictionary conforming to the schema in Section 6.9, with `requires_human_review` always set to true. This confirms the correctness of the mock implementation's contract compliance; it does not constitute an evaluation of live-LLM diagnostic accuracy, which is proposed as future evaluation in Section 8.9 and Section 10.7.

8.5 Human Review Testing

`src/test_reviewer.py` verifies that submitting a review decision (Accepted, Edited, or Rejected) correctly appends a new, well-formed row to `data/human_review_log.csv`,

including timestamp, case identifier, original diagnosis, decision, final diagnosis, and reviewer notes.

8.6 Streamlit UI Testing

The Streamlit application's four pages (Home, Diagnose Network, Human Review, Dashboard) were manually exercised to confirm that case selection, rule-check execution, diagnosis display, review submission, and dashboard rendering function correctly end-to-end, as a demonstration test scenario rather than an automated UI test suite.

8.7 Example Test Cases (Table 8.1)

Test ID	Module	Input	Expected Result	Observed Result	Status
TEST001	Rule Checker	Show output containing 'administratively down'	Flag Layer 1 interface issue, High severity	Issue flagged as expected	Pass
TEST002	Rule Checker	Show output with VLAN 10 missing from 'show vlan brief'	Flag Missing VLAN, Layer 2	Issue flagged as expected	Pass
TEST003	Rule Checker	Show output with mismatched gateway/subnet	Flag Gateway Mismatch, Layer 3	Issue flagged as expected	Pass
TEST004	Rule Checker	DHCP pool showing zero addresses available	Flag DHCP Pool Exhaustion, Layer 7	Issue flagged as expected	Pass
TEST005	Rule Checker	Routing table missing expected destination network	Flag Missing Route, Layer 3	Issue flagged as expected	Pass
TEST006	Rule Checker	ACL with no explicit permit before implicit deny	Flag ACL Implicit Deny, Layer 3/4	Issue flagged as expected	Pass
TEST007	Diagnosis Engine (Mock)	Rule finding: NAT translation not occurring	Structured diagnosis referencing NAT, Layer 3, human review required	Proposed test — demonstration test scenario (mock engine)	Proposed test
TEST008	Diagnosis Engine (Mock)	Rule finding: Wireless VLAN mismatch	Structured diagnosis referencing wireless VLAN mapping, Layer 2	Proposed test — demonstration test scenario (mock engine)	Proposed test

TEST001 through TEST006 correspond to deterministic rule-checker behaviour that was implemented and can be verified directly against known input strings. TEST007 and TEST008 are labelled as proposed/demonstration tests because, while the mock diagnosis engine can be exercised against these scenarios, a full evaluation of diagnostic quality in the sense

intended by the original live-AI architecture requires the LLM integration described in Section 10.7 and has not been claimed as a completed live-AI result.

8.8 Results and Discussion

The deterministic components of NetSage AI — the rule checker, the dataset, and the human-review logging mechanism — behaved consistently with their design specifications across the test scenarios exercised. The mock/rule-assisted diagnosis engine successfully produced schema-conformant structured output for all 36 dataset cases, confirming that the overall workflow (case selection through dashboard) functions end-to-end. As instructed, no claim is made regarding the diagnostic accuracy of a live LLM, since no live LLM calls were successfully completed during this project; any such evaluation is deferred to future work (Section 10.7) once the API quota constraint is resolved.

8.9 System Strengths

- Clear separation of deterministic and AI-oriented reasoning, aiding transparency and debuggability.
- Consistent schema-conformant output regardless of which diagnosis backend (mock or future live LLM) is in use.
- Mandatory human review baked into the data model (`requires_human_review` always true), not merely a UI convention.
- A dataset with balanced coverage across eight fault categories and five OSI layers.
- Complete audit trail of review decisions via the human review log.

8.10 System Limitations

- No live LLM diagnostic accuracy has been demonstrated; current results reflect rule-assisted mock output only.
- Test coverage for the diagnosis engine is limited to schema conformance rather than diagnostic correctness against unseen cases.
- UI testing was manual/demonstration-based rather than automated end-to-end testing.
- The dataset size (36 cases) is sufficient for demonstration but modest for rigorous statistical evaluation.

Chapter 9: Responsible AI and Human Oversight

9.1 Introduction

This chapter examines, in depth, why mandatory human oversight is central to the design of NetSage AI rather than an optional add-on. It discusses the risks of AI hallucination in technical domains, the role of evidence-based diagnosis in mitigating those risks, and presents illustrative demonstration examples of how the Accepted/Edited/Rejected review workflow is intended to function.

9.2 Why Human Review is Necessary

Network configuration changes made on the basis of an incorrect diagnosis can cause further outages or security exposures, particularly in production environments. Even a well-designed AI or rule-based system can reach an incorrect conclusion — a rule checker may match a superficially similar pattern for the wrong reason, and a generative model may produce a fluent but factually ungrounded explanation. Mandatory human review ensures a qualified engineer verifies every recommendation before it is trusted or acted upon, which is especially important for junior engineers who are still building the experience needed to recognise a flawed diagnosis independently.

9.3 Risks of AI Hallucination

In generative AI systems, hallucination refers to the production of confident, plausible-sounding output that is not actually supported by the input data. In a network-troubleshooting context, a hallucinated diagnosis might reference a configuration element that does not appear anywhere in the supplied show-command output, or recommend a fix step unrelated to the actual evidence. NetSage AI's design mitigates this risk structurally: the diagnosis schema requires an explicit evidence field grounded in rule-checker findings, and the mock implementation currently derives its output directly from those findings rather than from unconstrained generation, a discipline that the prompt in Section 7.8 is designed to preserve once a live LLM is integrated.

9.4 Evidence-Based Diagnosis

Every diagnosis produced by NetSage AI is required to carry an evidence field referencing the specific show-output text or rule-checker finding that supports the stated root cause. This design choice allows a human reviewer to quickly verify or refute a diagnosis by comparing the stated evidence against the actual case data, rather than having to trust the diagnosis output at face value.

9.5 Confidence Limitations

The diagnosis schema includes a confidence field intended to convey the system's own estimate of certainty. In the current mock/rule-assisted implementation, this value is derived heuristically from rule-match strength rather than from a calibrated probabilistic model, and should therefore be interpreted as an indicative signal rather than a statistically validated confidence measure. This limitation is disclosed here so that reviewers do not over-interpret the confidence score in its present form.

9.6 Accept/Edit/Reject Workflow

As described in Section 6.10, every diagnosis passes through a three-way human decision: Accepted (the diagnosis is correct as stated), Edited (the diagnosis is partially correct and is corrected by the reviewer), or Rejected (the diagnosis is incorrect). This structure ensures that even a partially useful diagnosis can be captured and corrected rather than being discarded outright, while still preventing an incorrect diagnosis from being treated as final.

9.7 Human Review Logging

Every review decision, together with the original diagnosis and any corrected final diagnosis, is permanently logged in `data/human_review_log.csv` (Section 6.8). This log is the foundation for the Responsible AI analytics proposed in Section 4.11 and Section 10.7, including future acceptance-rate and AI-human agreement measurement.

9.8 Example Corrected Diagnosis Cases

The following five examples are demonstration examples for Responsible AI evaluation, illustrating the intended Accept/Edit/Reject workflow using the mock diagnosis engine's output style. They are not claimed to be outputs of a live large language model.

Demonstration Example 1

Field	Value
Case ID	CASE003 (illustrative)
Network Problem	PC cannot obtain an IP address via DHCP.
AI/Mock Diagnosis	Root cause: DHCP pool exhaustion; OSI Layer 7.
Human Decision	Accepted
Corrected Diagnosis	Not applicable — diagnosis accepted as stated.
Reason for Correction	Not applicable.
Lesson Learned	Confirms the rule-checker signature for DHCP pool exhaustion aligned with the reviewer's own assessment.

Demonstration Example 2

Field	Value
Case ID	CASE011 (illustrative)
Network Problem	Two hosts in the same VLAN cannot communicate.
AI/Mock Diagnosis	Root cause: Missing VLAN; OSI Layer 2.
Human Decision	Edited
Corrected Diagnosis	VLAN present but not permitted on the trunk link (VLAN not allowed on trunk).
Reason for Correction	The rule checker matched a VLAN-related keyword but the more specific trunk-allowed-VLAN condition better explained the evidence.
Lesson Learned	Highlights the importance of distinguishing closely related VLAN fault sub-types and refining rule specificity.

Demonstration Example 3

Field	Value
Case ID	CASE019 (illustrative)
Network Problem	Remote network unreachable from a branch router.
AI/Mock Diagnosis	Root cause: Incorrect next-hop; OSI Layer 3.
Human Decision	Rejected
Corrected Diagnosis	Missing static route to the destination network entirely (Missing Route), not an incorrect next-hop on an existing route.
Reason for Correction	The rule checker's evidence text was ambiguous between the two closely related routing fault patterns.
Lesson Learned	Demonstrates why mandatory human review is essential — a plausible but incorrect diagnosis was caught before being treated as final.

Demonstration Example 4

Field	Value
Case ID	CASE026 (illustrative)
Network Problem	Internal host cannot access an external website.
AI/Mock Diagnosis	Root cause: NAT translation not occurring; OSI Layer 3.
Human Decision	Accepted
Corrected Diagnosis	Not applicable — diagnosis accepted as stated.
Reason for Correction	Not applicable.
Lesson Learned	Confirms correct evidence linkage between the show-output NAT translation table and the stated root cause.

Demonstration Example 5

Field	Value
Case ID	CASE033 (illustrative)
Network Problem	Wireless client cannot reach a wired server on a different VLAN.
AI/Mock Diagnosis	Root cause: Wireless VLAN mapping mismatch; OSI Layer 2.
Human Decision	Edited
Corrected Diagnosis	Wireless VLAN mapping correct, but the wired-side trunk did not permit the wireless VLAN (VLAN not allowed on trunk).
Reason for Correction	The initial diagnosis correctly identified the general fault category but attributed it to the wrong side of the connection.
Lesson Learned	Illustrates the value of the Edited category for partially correct diagnoses, preserving useful signal while correcting the specific attribution.

9.9 Responsible AI Benefits

- Reduces the risk of learners or engineers acting on an unverified, potentially incorrect diagnosis.
- Builds an audit trail that can later be used to measure and improve system reliability (AI-human agreement analytics, Section 10.7).
- Reinforces good troubleshooting discipline by requiring evidence for every conclusion.
- Provides a template for how future live-LLM integration can be introduced without compromising oversight.

9.10 Ethical Considerations

Beyond technical correctness, this project raises broader ethical considerations relevant to applying AI in technical education and operations: the importance of not overstating a system's capabilities (hence the transparent disclosure of the mock/rule-assisted implementation throughout this report), the responsibility to avoid encouraging blind trust in AI-generated technical recommendations, and the value of designing systems whose limitations are visible and correctable rather than hidden behind a confident-sounding interface.

Chapter 10: Conclusion and Future Scope

10.1 Conclusion

NetSage AI demonstrates that a combination of deterministic rule-based logic, a structured AI-oriented diagnosis architecture, and mandatory human review can provide a transparent, evidence-grounded approach to Cisco-style network troubleshooting, even when a live large language model backend is not available. By honestly substituting a mock/rule-assisted diagnosis engine in place of the originally planned live API integration, and preserving the same architectural contract, the project delivers a fully functional end-to-end system while remaining truthful about the current stage of AI integration.

10.2 Summary of Work

The project delivered a 36-case troubleshooting dataset spanning eight networking fault categories and five OSI layers; a deterministic Python rule checker covering eleven configuration anti-patterns; a structured diagnosis engine (mock/rule-assisted, with a schema designed for future LLM integration); a mandatory human review workflow with Accepted/Edited/Rejected outcomes and persistent logging; and a four-module Streamlit web application (Home, Diagnose Network, Human Review, Dashboard).

10.3 Project Achievements

- A working, testable end-to-end troubleshooting workflow from case selection to human-reviewed, logged diagnosis.
- A dataset and rule checker aligned closely with Cisco Networking Academy troubleshooting concepts.
- A Responsible AI architecture in which human oversight is structurally mandatory rather than optional.
- Transparent handling and documentation of the API quota limitation, including a functioning substitute implementation.

10.4 Technical Contributions

The project contributes a reusable structured JSON schema for evidence-grounded network diagnosis, a modular rule-checker design that can be extended with additional configuration

anti-patterns, and a human-review logging schema suitable for future Responsible AI analytics such as AI-human agreement measurement.

10.5 Practical Contributions

As an educational tool, NetSage AI offers junior network engineers and students a structured way to practise connecting symptoms to root causes across common Cisco fault categories, while reinforcing the discipline of verifying rather than blindly trusting AI-generated technical output.

10.6 Current Limitations

(Table 10.1) The current implementation has the limitations summarised below, none of which are concealed elsewhere in this report.

Limitation	Description
Mock Diagnosis Engine	A live OpenAI-compatible API could not be demonstrated due to an insufficient_quota error; a mock/rule-assisted engine is used instead.
Dataset Scope	The 36-case dataset is based on structured/simulated lab scenarios rather than live network captures.
Simplified Show Output	Show-command outputs are simplified Cisco-style examples rather than raw device output.
Rule Coverage	The rule checker covers a limited set of eleven deterministic configuration patterns.
Storage	CSV files are used instead of a production-level database.
Device Integration	No real-time integration with physical network devices is implemented.
Packet Tracer Integration	Packet Tracer topology integration is currently conceptual/manual rather than automated.
Evaluation Metrics	Advanced AI evaluation metrics require real LLM outputs and larger-scale testing, not yet performed.

10.7 Future Scope

(Table 10.2) The following enhancements are proposed as future work, building directly on the current architecture without requiring a redesign of its core contracts.

Future Enhancement	Description
Real LLM API Integration	Replace the mock diagnosis engine with a live LLM call using the existing prompt (Section 7.8) and schema (Section 6.9).

Future Enhancement	Description
Retrieval-Augmented Generation (RAG)	Ground diagnoses in retrieved Cisco documentation to further reduce hallucination risk.
Cisco Documentation Knowledge Base	Build a structured knowledge base to support RAG and evidence citation.
Automated Show-Output Parsing	Parse real (rather than simplified) Cisco show-command output automatically.
Network Device Integration	Integrate with live or lab network devices for real-time data collection.
SNMP Integration	Collect device health and configuration data via SNMP.
Syslog Analysis	Incorporate syslog data as an additional diagnostic evidence source.
Real-Time Monitoring	Extend the dashboard toward real-time network health monitoring.
Predictive Fault Detection	Explore predictive analytics to flag likely faults before they cause outages.
Automated Topology Visualization	Generate topology diagrams automatically from device data.
Database Migration (SQLite/PostgreSQL)	Replace CSV storage with a relational database for scalability.
User Authentication	Add login and identity management for multi-user deployments.
Multiple Reviewers	Support multiple concurrent human reviewers per case.
Role-Based Access	Differentiate permissions between engineers, reviewers, and administrators.
AI-Human Agreement Analytics	Compute acceptance/edit/rejection rates and AI-human agreement from the review log.
Historical Knowledge Base	Build a searchable history of past diagnoses and corrections.
Additional Packet Tracer Scenarios	Expand the dataset with further lab-derived cases.
More Advanced Deterministic Checks	Extend the rule checker beyond the current eleven patterns.

10.8 Final Remarks

NetSage AI illustrates that meaningful applied-AI systems for technical education need not depend on a fully live generative backend to demonstrate sound architecture, evidence-based reasoning, and responsible human oversight. By transparently documenting the API quota constraint encountered during development and adapting the system accordingly, this project aims to model the kind of honest, limitation-aware engineering practice that Responsible AI principles call for, while leaving a clear and well-documented path toward full live LLM integration in future work.

References

The following reference categories are provided as a template. Only sources genuinely consulted and verified should be cited with full bibliographic detail in the final submitted report; no journal articles, authors, DOIs, or URLs have been fabricated for this report. Entries below marked [TO BE COMPLETED] should be filled in with the specific documentation pages actually consulted.

Cisco Networking Academy Materials

[TO BE COMPLETED — e.g., specific Cisco Networking Academy course module(s) consulted, with course title and access date.]

Cisco Networking Documentation

[TO BE COMPLETED — e.g., Cisco IOS command reference pages consulted, with exact title and access date.]

Python Documentation

[TO BE COMPLETED — Python Software Foundation, Python 3 documentation, <https://docs.python.org/3/> — verify exact pages used.]

Streamlit Documentation

[TO BE COMPLETED — Streamlit Inc., Streamlit documentation, <https://docs.streamlit.io/> — verify exact pages used.]

Pandas Documentation

[TO BE COMPLETED — pandas development team, pandas documentation, <https://pandas.pydata.org/docs/> — verify exact pages used.]

OpenAI API Documentation

[TO BE COMPLETED — OpenAI, API reference documentation — verify exact pages used, including rate limit / quota documentation relevant to Section 7.13.]

Responsible AI Documentation

[TO BE COMPLETED — organisational or academic Responsible AI guidance consulted, if any, with full citation.]

Appendices

Appendix A: Sample Dataset Records

case_id	concept	osi_layer	severity	expected_fault
CASE001	VLAN	Layer 2	High	Missing VLAN
CASE006	Default Gateway	Layer 3	High	Gateway Mismatch
CASE011	DHCP	Layer 7	Medium	DHCP Pool Exhaustion
CASE016	DNS	Layer 7	Medium	DNS Resolution Failure
CASE021	Routing	Layer 3	High	Missing Route
CASE026	ACL	Layer 3/4	High	ACL Implicit Deny
CASE031	NAT	Layer 3	Medium	NAT Translation Failure
CASE034	Wireless	Layer 2	Medium	Wireless VLAN Mismatch

Appendix B: Troubleshooting Categories

- VLAN
- Default Gateway
- DHCP
- DNS
- Routing
- ACL
- NAT
- Wireless Networking

Appendix C: Rule Checker Logic (Summary)

20. Interface administratively down — matches 'administratively down' in show output.
21. Missing VLAN — VLAN referenced in topology note absent from 'show vlan brief'.
22. VLAN not allowed on trunk — VLAN present but excluded from trunk allowed list.
23. Missing default gateway — no default gateway configured where required.
24. Gateway mismatch — configured gateway inconsistent with subnet.
25. Missing route — expected destination network absent from routing table.
26. Incorrect next-hop — route present but next-hop address incorrect.
27. DHCP pool exhaustion — DHCP pool shows zero available addresses.

28. Missing DHCP relay — DHCP relay/helper address absent where required.
29. ACL implicit deny — no matching permit before the implicit deny.
30. Overly permissive ACL — ACL permits broader traffic than intended.

Appendix D: Diagnosis JSON Structure

```
{
  "root_cause": "",
  "osi_layer": "",
  "confidence": 0,
  "evidence": [],
  "next_command": "",
  "fix_steps": [],
  "requires_human_review": true
}
```

Appendix E: Human Review Log Structure

```
timestamp, case_id, ai_diagnosis, human_decision, final_diagnosis, reviewer_notes
```

Appendix F: Sample Installation Instructions

```
git clone <repository-url>
cd NetSage-AI
python -m venv venv
venv\Scripts\activate      (Windows) OR  source venv/bin/activate  (macOS/Linux)
pip install -r requirements.txt
streamlit run app.py
```

Appendix G: Project Folder Structure

```
NetSage-AI/
├── data/ (cases.csv, human_review_log.csv)
├── prompts/ (diagnose_prompt.md)
├── src/ (rule_checker.py, ai_diagnosis.py, reviewer.py, tests)
├── examples/
├── venv/
├── app.py, .env, .gitignore, requirements.txt
```

Appendix H: Screenshots to Include

[INSERT SCREENSHOT: Streamlit Home Page]

[INSERT SCREENSHOT: Diagnose Network Page]

[INSERT SCREENSHOT: Rule Checker Output]

[INSERT SCREENSHOT: Diagnosis Result]

[INSERT SCREENSHOT: Human Review Page]

[INSERT SCREENSHOT: Dashboard]

[INSERT SCREENSHOT: Terminal Testing Output]